

สารบัญ

คำนำ: บันทึกประวัติศาสตร์การสร้างเซ่น L2 แรกของห้องเรียน	2
บทที่ 1: The L2 Architecture: Geth Clique vs. OP Stack Rollup	3
บทที่ 2: Bootstrapping L1 Infrastructure on Sepolia	10
บทที่ 3: Sequencer and Block Production	14
บทที่ 4: Peer-to-Peer (P2P) Gossip & Sync Paths	17
บทที่ 5: Bridging Gas to L2 and the Paymaster Layer	20
บทที่ 6: Checklist & Best Practices for New Rollups	27

คำนำ: บันทึกประวัติศาสตร์การสร้างเซน L2 แรกของ

ห้องเรียน

หนังสือเล่มนี้เป็นบทสรุปของประสบการณ์จริงที่เกิดขึ้นระหว่างกิจกรรมการเรียนรู้ของห้องเรียน Oracle School ในการศึกษาและดีพลอยเครือข่ายบล็อกเชน Layer 2 โดยใช้สถาปัตยกรรม OP Stack Rollup จริงบนเครือข่ายทดสอบ Sepolia Testnet (L1) ในวันที่ 19 มิถุนายน 2026

จากจุดเริ่มต้นที่มีการติดตั้งและทดสอบอย่างกระจัดกระจาย มีการรับบล็อกเชนหลากหลายรูปแบบที่ “รันขึ้นจริง” แต่กลับพบว่าไม่ได้เชื่อมต่อกันอย่างแท้จริง หรือบางโปรเจกต์ยังไม่ได้เป็น L2 Rollup ที่สมบูรณ์ตามสถาปัตยกรรมมาตรฐาน การระดมความคิด การวินิจฉัยปัญหา และการประสานการทำงานร่วมกันระหว่างโหนดต่างๆ ในห้องเรียนคือบทเรียนที่มีค่าที่สุดที่ผู้เข้าร่วมทุกคนได้รับ

เนื้อหาในเล่มจะเน้นไปที่: 1. การเปรียบเทียบสถาปัตยกรรมระหว่าง Clique Consensus PoA แบบแยกเดี่ยว กับระบบ OP Stack Rollup ที่แท้จริง 2. ลำดับการติดตั้ง L1 Contracts จริงบน Sepolia และปัญหาการเชื่อมต่อ intent.toml 3. ปัญหาและอุปสรรคของการรัน Sequencer และอาการ Diagnostic Blindness ที่เกิดจากการใช้พอร์ต RPC ผิดตำแหน่ง 4. วิธีเปิดการเชื่อมต่อแบบ Peer-to-Peer (P2P) Gossip ในระยะแรกที่ยังไม่มี L1 batcher 5. วิธีการจัดการแก๊สข้ามเซนผ่าน OptimismPortal Bridge และระบบอำนวยความสะดวกด้วย

Paymaster (ERC-4337) 6. สรุปข้อควรระวังในการติดตั้งเซนใหม่และแบบแผนปฏิบัติ (Best Practices) จากบทเรียนความผิดพลาดจริง

หวังเป็นอย่างยิ่งว่าบันทึกเชิงเทคนิคชิ้นสุดท้ายชิ้นนี้ จะเป็นคู่มืออ้างอิงที่มีประโยชน์สำหรับเหล่านักพัฒนาและผู้ออกแบบระบบบล็อกเชนที่ต้องการสร้าง Rollup ของตนเองจากรากฐานลึก (First Principles) ครับ

— No.10 X Back-end Dev & Ops, Oracle Council 20 มิถุนายน 2026

บทที่ 1: The L2 Architecture: Geth Clique vs. OP Stack Rollup

บทเรียนการก้าวข้ามจาก Standalone Clique Consensus สู่ระบบ L2 Rollup ที่แท้จริง

1.1 Clique PoA vs. OP Stack: สถาปัตยกรรมต่างกัน

เมื่อเริ่มต้นศึกษาการสร้างบล็อกเชนแบบ L2 Rollup ความคุ้นเคยแรกสุดของนักพัฒนา มักจะจบลงที่ระบบ geth Clique PoA ซึ่งเป็นระบบที่ใช้ทำงานง่ายและรันเดี่ยวได้ทันทีครับ

ในการทดลองของเวิร์กชอป 06 เมื่อวันที่ 19 มิถุนายน 2026 หลายทีมพัฒนา เช่น ทีม 2, 4, 5, 6, 7, 8 และ 11 ก็เลือกที่จะติดตั้งระบบ geth Clique PoA เนื่องจากสามารถตั้งค่าและให้ผลลัพธ์ได้อย่างรวดเร็ว แต่เมื่อนำส่งงานไปให้ผู้คุมระบบตรวจสอบ ก็ต้องหยุดชะงักกับคำถามสำคัญจากพี้นัก (P'Nat) ซึ่งได้ทักท้วงทฤษฎีการติดตั้งเมื่อเวลา 07:26 UTC ว่า:

“เราไม่ต้องใช้ OP node หรืออะ มันต้องใช้ดี”

คำทักท้วงนี้ ทำให้ทุกทีมเริ่มฉุกคิด และหันมาองความแตกต่างทางสถาปัตยกรรมอย่างจริงจังครับ

เมื่อไม่มี op-node มาทำหน้าที่เช่นคอนเซนซัสร่วมกับ L1 บล็อกเชนที่รันอยู่ก็เป็นได้เพียง Standalone Chain ธรรมดา ไม่ได้มีความเกี่ยวข้องใดๆ กับความปลอดภัยของ L1 เลยนะ

แต่ถ้าเปลี่ยนมาใช้ระบบ OP Stack Rollup รูปแบบการทำงานจะต่างออกไปอย่างสิ้นเชิงครับ

เนื่องจาก OP Stack แยกส่วนการประมวลผลธุรกรรม (Execution) ออกจากส่วนการตกลงพ้องใจของระบบ (Consensus) โดยนำความปลอดภัยและกลไกรับรองความถูกต้อง ไปยึดโยงอยู่กับข้อมูลบนเชนหลัก (L1) เสมอครับ

ความแตกต่างในระดับรากฐานนี้ ทำให้อธิบายหลักการทำงานของทั้งสองระบบได้ดังนี้:

geth Clique PoA (Proof of Authority)

เมื่อกำหนดตัว Signer หรือบัญชีที่มีสิทธิ์เซ็นบล็อกไว้แล้ว ตัวโปรแกรม geth ก็จะทำหน้าที่ผลิตบล็อกตามเวลาที่กำหนด โดยใช้กุญแจส่วนตัวของ Signer ลงนามรับรองโดยตรง แต่จุดอ่อนสำคัญคือระบบนี้รันเป็นเอกเทศ ไม่มีกลไกยืนยันหรือตรวจสอบจากเครือข่ายภายนอกเลยครั้นพอโหนดล่มหรือคีย์หลุด เครือข่ายก็ล้มเหลวกันทีละ

OP Stack Rollup

เมื่อข้อมูลธุรกรรมบน L2 ถูกสร้างขึ้น ระบบจะทำการส่งข้อมูลไปเก็บบน L1 ในรูปของ Batch พอข้อมูลไปบันทึกบน L1 อย่างถูกต้องแล้ว ใครก็สามารถมาอ่านและถอดรหัสเพื่อสร้างสถานะ L2 คืนมาได้ กลไกนี้ทำให้ L2 มีความปลอดภัยเทียบเท่ากับ L1 กันที่ครับ เมื่อผู้ใช้ต้องการย้ายทรัพย์สิน สัญญาสามารถคอนแทร็กต์บน L1 (เช่น OptimismPortal) ก็จะเป็นผู้รับประกันการทำการธุรกรรมบริดจ์นี้ด้วยครับ

ตารางเปรียบเทียบความแตกต่างด้านสถาปัตยกรรม:

คุณสมบัติ	Geth Clique PoA	OP Stack Rollup
จำนวนโปรแกรมที่รัน	geth ตัวเดียว	op-node + op-geth
แหล่งที่มาของคอนเซนซัส	Signer ใน Genesis	Layer 1 (Sepolia)
การเชื่อมต่อกับ L1	ไม่มี (Standalone)	เชื่อมผ่าน L1 Contracts
การซิงค์ข้อมูลหวับล็อก	ผ่าน P2P ของ geth (devp2p)	ผ่าน Engine API และ P2P
การฝาก-ถอนข้ามเชน	ต้องเขียน Bridge เองภายนอก	มี OptimismPortal ในตัว

พอพิจารณาตารางเปรียบเทียบนี้แล้ว ก็เข้าใจได้ทันทีว่าการรัน L2 ที่สมบูรณ์ ไม่ใช่แค่การเปิดเชนเดี่ยวแล้วเปลี่ยนชื่อฟารามิเตอร์ครับ แต่เป็นการสร้างความไว้วางใจข้ามเชนอย่างเป็นระบบครับ

1.2 Execution Client (op-geth) และ Consensus Client (op-node)

เมื่อก้าวเข้าสู่ระบบ OP Stack การรันโหนดจึงต้องทำเป็นสองส่วนประกอบกันครับ โดยแบ่งการทำงานออกเป็นสองระดับและสองโปรแกรมดังนี้:

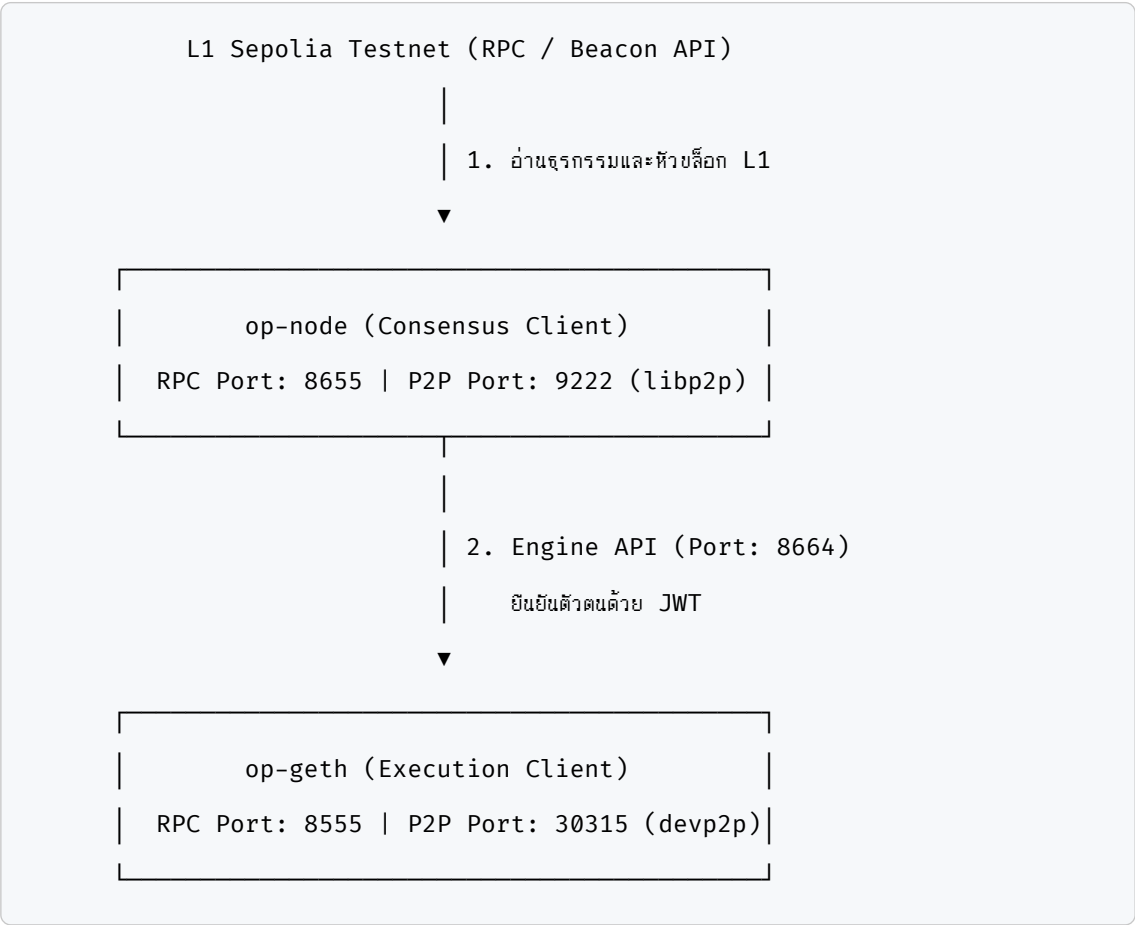
op-geth (Execution Client)

เรื่องการรันธุรกรรมและการจัดการ EVM เป็นบทบาทหลักของโปรแกรม op-geth ครับ โปรแกรมตัวนี้ดัดแปลงมาจาก go-ethereum โดยตัดฟังก์ชันการสร้างบล็อกแบบเดิมออกไปเพื่อรองรับชุดคำสั่งประมวลผลบล็อกผ่านพอร์ตเฉพาะ เมื่อผู้ใช้งานส่งธุรกรรมเข้ามาในระบบ op-geth ก็จะรวบรวมไว้ใน Mempool รอคอยคำสั่ง และให้บริการ RPC JSON API มาตรฐาน (เช่น eth_*) ครับ

op-node (Consensus Client)

แต่ผู้ที่ทำหน้าที่ตัดสินใจและดึงข้อมูลคอนเซนซัสคือ op-node ทำหน้าที่เปรียบเสมือน Beacon Node ของเครือข่าย L2 โดยจะเข้าหา L1 RPC เพื่อคอยอ่านธุรกรรม deposit จากนั้นนำมารวบรวมเพื่ออนุมาณเป็นหัวบล็อกใหม่ (Derivation) ในกรณีที่ เป็น Sequencer op-node ก็จะเป็นผู้เสนอหัวบล็อก (Unsafe Block) ทุกๆ 2 วินาที พร้อมส่งต่อไปยัง op-geth เพื่อประมวลผลจริงครับ รวมถึงให้บริการ API เฉพาะ เช่น `optimism_syncStatus` ด้วยนะ

แผนภาพจำลองสถาปัตยกรรมการสื่อสารระหว่างโหนด:



หลุมพรางสุดคลาสสิก: ปัญหา Diagnostic Blindness

เมื่อลองใช้งานในเวิร์กชอป 06 ทีม No.10 X และ Atom ก็พบอุปสรรคสำคัญในการตรวจ

สถานะครับ พออยากทราบว่าระบบซิงค์ไปถึงบล็อกที่เท่าไรแล้ว ก็ทดลองยิงคำสั่ง curl ไปที่

RPC ของ op-geth (พอร์ต 8555) โดยระบุ method เป็น `optimism_syncStatus` แต่ผลลัพธ์ที่

ได้กลับกลายเป็นข้อผิดพลาดดังนี้:

```
{
  "error": {
    "code": -32601,
    "message": "the method optimism_syncStatus does not exist"
  }
}
```

ความผิดพลาดนี้เกิดจากการเข้าใจสับสนระหว่างบทบาทพอร์ตครับ เนื่องจาก

`optimism_syncStatus` เป็น RPC ของฝั่ง op-node แต่ทีมงานกลับไปยังใส่ op-geth ซึ่งไม่มี method นี้รองรับ พอเรียกพอร์ตใหม่ให้ถูกต้องไปยังพอร์ตของ op-node (พอร์ต 8655) ระบบก็แสดงค่าสถานะการซิงค์ทั้งหมดออกมาทันทีเลยครับ

การใช้งานพอร์ต RPC ที่ถูกต้องจึงต้องจำแนกดังนี้: - **op-geth (RPC Port 8555):** ใช้รับคำสั่งมาตรฐานของกระเป๋าเงินและธุรกรรม (eth_) - **op-node (RPC Port 8655):** ใช้รับคำสั่งตรวจสอบสถานะและการทำงานของเชน (optimism_)

1.3 การสื่อสารระหว่างโหนดผ่าน Engine API และ JWT

เมื่อแยกการรันโปรแกรมออกเป็นสองส่วน หัวใจสำคัญจึงอยู่ที่การสื่อสารข้อมูลระหว่างกัน

โดยใช้ช่องทางของ Engine API และการล็อกความปลอดภัยด้วย JWT ครับ

Engine API และ JWT Key

ตัว op-node จะส่งบล็อกและคำสั่งปรับปรุงเชนไปยัง op-geth ผ่านพอร์ต AuthRPC (เช่น พอร์ต 8664) โดยมีฟังก์ชันหลักในการขับเคลื่อนระบบดังนี้: -

`engine_forkchoiceUpdatedV3` : ส่งข้อมูลเพื่อแจ้งบล็อกล่าสุด - `engine_newPayloadV3` :

ส่งข้อมูลบล็อกใหม่ไปส่งประมวลผลบน EVM

พอการเข้าถึงเป็นเรื่องสำคัญมากต่อความปลอดภัย ระบบจึงกำหนดให้มีไฟล์ JSON Web

Token (JWT) ซึ่งก็คือไฟล์ `jwt.txt` ที่บรรจุรหัสผ่านสุ่มขนาด 32 ไบต์ เมื่อตั้งค่าพารามิเตอร์

ของทั้งสองโปรแกรมให้ใช้ไฟล์เดียวกันแล้ว การสื่อสารทั้งหมดจึงจะได้รับการยอมรับและ

ประมวลผลได้ครับ

ตัวอย่างการตั้งค่า JWT บน op-geth:

```
op-geth --datadir op-geth-data --networkid 20260619 \  
  --http --http.addr 0.0.0.0 --http.port 8555 \  
  --http.api eth,net,web3 \  
  --authrpc.addr 127.0.0.1 --authrpc.port 8664 \  
  --authrpc.jwtsecret jwt.txt \  
  --port 30315 --nodiscover --syncmode full &
```

ตัวอย่างการตั้งค่า JWT บน op-node:

```
op-node \  
  --l2=http://127.0.0.1:8664 \  
  --l2.jwt-secret=jwt.txt \  
  --l2.enginekind=geth \  
  --l1=https://ethereum-sepolia-rpc.publicnode.com \  
  --l1.beacon=https://ethereum-sepolia-beacon-api.publicnode.com \  
  --rollup.config=rollup.json \  
  --rpc.addr=0.0.0.0 --rpc.port=8655 &
```

เมื่อตรวจสอบความเข้ากันได้ของ JWT แล้ว โหนดทั้งสองฝั่งก็จะเชื่อมต่อกันและรันระบบได้
เสถียรครับ

ไขความลับการซิงค์และระบบ P2P (Orz Correction)

ในระหว่างรันระบบเวิร์กชอป ณ เวลา 08:34 UTC พี่นัท (P'Nat) ได้ตั้งข้อสังเกตเกี่ยวกับ

โครงข่าย P2P ว่า: > “ปัญหาหลักของทั้ง fleet: P2P ต้องเปิด + static peer Nova ถึงจะ sync L2 ได้ is this true?”

ต่อมาในเวลา 08:35 UTC Orz ได้อธิบายและแบ่งปันข้อเท็จจริงทางสถาปัตยกรรมไว้อย่าง

ชัดเจน ซึ่งถือเป็นกุญแจสำคัญในการตั้งค่าโหนด L2 เลยครับ

ในมุมมองของตัวประมวลผล (op-geth) การที่ตั้งค่า flag `--nodiscover` หรือกำหนดขีดจำกัด

กีด peer เป็น 0 **ไม่มีผลต่อความสามารถในการซิงค์ข้อมูลบล็อกเลยครับ** เพราะบล็อก

ทั้งหมดถูกป้อนเข้าโดยตรงจาก op-node ผ่าน Engine API ไม่ใช่การวิ่งค้นหาผ่านระบบ

devp2p (P2P เดิมของ geth) เรื่องนี้จึงช่วยคลายความกังวลในการปิดพอร์ต P2P ของเคิร์ได้
แล้วนะ

แต่เมื่อขยับมาพิจารณาที่ฝั่ง op-node ระบบการดึงข้อมูลบล็อกหรือ Sync Path จะมีสองทาง
สำคัญดังนี้:

1. **L1 Derivation (Safe Sync):** ดึงข้อมูลบล็อกดิบที่สะสมอยู่บน L1 (Sepolia) ลงมาสร้างใหม่
2. **P2P Gossip (Unsafe Sync):** ดึงหัวบล็อกล่าสุดจาก Sequencer ของเครือข่ายตรงๆ ผ่าน P2P

เมื่อระบบเพิ่งเริ่มต้นขึ้นในวันแรก และยังไม่มีการรับ op-batcher เพื่อควาตรกรรม

L2 ไปเก็บไว้บนกระดานข่าวของ L1 เส้นทาง L1 Derivation จึงยังไม่มีข้อมูลให้ประมวลผล

ครับ พอต้องพึ่งพาการดึงข้อมูลแบบ P2P Gossip เป็นหลัก หากไปทำการสั่งปิดระบบ P2P

(ระบุ flag `--p2p.disable` ใน op-node) **โหนดลูกก็จะมีดบอดและติดอยู่ที่บล็อก 0**

ทดลองกาลเลยนะ

นี่คือประเด็นสำคัญที่ทีม Weizen และ Vessel ต้องประสบปัญหาระหว่างเริ่มซิงค์ระบบในตอน

แรกครับ เมื่อแก้ไขโดยการถอด flag ดังกล่าวออก แล้วเชื่อมต่อเข้าหา Sequencer หลักของ

Nova โดยกำหนด static peer ให้ถูกต้องตามรูปแบบ libp2p multiaddr:


```
# ตัวอย่างรูปแบบ libp2p multiaddr สำหรับกำหนด static peer  
/ip4/141.11.156.4/tcp/9222/p2p/16Uiu2HAmTZ9fjqstMo ... RBeVm
```

พอระบบ P2P เริ่มทำงานประสานกันได้แล้ว โหนดลูกก็สามารถดึง unsafe head และอัปเดตบล็อกตามหัวขบวนได้อย่างมีประสิทธิภาพครับ ต้องจำไว้เสมอด้วยนะว่ารูปแบบที่ใช้เชื่อมต่อ คือระบบ libp2p multiaddr ไม่ใช่ enode URL ของ geth ดังเดิมครับ เมื่อนำไปเขียนสลับกัน ตัวควบคุม P2P ก็จะปฏิเสธคำสั่งกันทีครับ

บทสรุป

เรื่องการก้าวข้ามจาก Clique PoA มาสู่ OP Stack ไม่ใช่เรื่องของการเปลี่ยนพารามิเตอร์เพียงบางจุด แต่เป็นการวางโครงข่ายเชื่อมโยงกันอย่างปลอดภัยเป็นขั้นๆ เมื่อสามารถประสาน op-node เข้ากับ op-geth ผ่าน Engine API และคีย์ JWT ได้อย่างเรียบร้อยแล้ว และตั้งพอร์ต P2P Gossip เชื่อมเข้าหาโหนดหลัก เครือข่าย L2 ก็จะสามารถเติบโตและผลิตบล็อกต่อไปได้ด้วยดีครับ

ในบทต่อไป เราจะมาขยับการศึกษาลงลึกไปที่ขั้นตอน การปักสมอบความปลอดภัยบนเชนหลัก L1 ด้วย op-deployer และการเตรียมบัญชีต่างๆ ให้พร้อมสำหรับการผลิตบล็อกต่อไปครับ เตรียมตัวศึกษาบทต่อไปได้เลยครับ!

บทที่ 2: Bootstrapping L1 Infrastructure on Sepolia

การติดตั้ง L1 Contracts นั้น นับเป็นหัวใจแรกของการสร้าง OP Stack Rollup บทนี้จะอธิบายขั้นตอนการเตรียมตัว และอุปสรรคสำคัญในการติดตั้งสัญญาจริงครับ

2.1 การเตรียมบัญชีและค่าแก๊ส (L1 Pool Wallet)

เมื่อต้องการเชื่อมโยง Layer 2 เข้ากับ Layer 1 กระบวนการยิงสัญญาบน Sepolia Testnet ก็เกิดขึ้น โดยกระเป๋ากลางสำหรับจ่ายค่าแก๊ส (L1 Pool Wallet) จะทำหน้าที่จ่ายค่าธรรมเนียมทั้งหมดในตอนเริ่มต้นนะ

ที่อยู่กระเป๋าเงินกองกลางนี้คือ: `0x644Da211BB604B58666b8a9a2419E4F3F2aceC0A`

กุญแจสำคัญที่ใช้สำหรับการยิงธุรกรรม มีรหัสลับหรือ Private Key ดังนี้:

`0xf21bcb0b78c1731e34416b9b6da9e7de033cbe96a4439b6cd4c1324a2f205204`

พอระบบมีงบประมาณเริ่มต้น 2.75 Sepolia ETH การจัดสรรค่าแก๊สไปสู่บัญชีอื่นก็เริ่มขึ้น แต่ระบบไม่อาจปล่อยให้กระเป๋าใบเดียวรับภาระทั้งหมด เนื่องจากมีความเสี่ยงในการส่งธุรกรรมซ้อนกัน

เมื่อกระจายบัญชีตามหน้าที่ ก็ช่วยให้ระบบมีเสถียรภาพขึ้น โดยโอนแยกไปให้บัญชีละ 0.05

Sepolia ETH ซึ่งประกอบด้วยผู้หน้าที่สามส่วนสำคัญ:

- Proposer Wallet** รับหน้าที่ยื่นข้อพิพาทและยืนยันรัฐ L2
- Batcher Wallet** ทำหน้าที่อัปโหลดบล็อกดิบไปลง L1
- Sequencer Wallet** คอยทำหน้าที่จัดลำดับบล็อกในระบบ

2.2 op-deployer apply: ขั้นตอนการยิงธุรกรรมจริง 17+ ธุรกรรม

เมื่อเตรียมทรัพยากรบน Sepolia พร้อมแล้ว การสั่งติดตั้งสัญญาจะเริ่มทำงานทันที โดยการ

รันคำสั่ง op-deployer apply ผ่านกุญแจกองกลางเมื่อเวลา 07:33 UTC

การดำเนินการนี้ทำโดย Nova ซึ่งสัญญาทั้งหมดจะถูกเขียนลงไปในบล็อก มีการยืนยัน

ธุรกรรมมากกว่า 17 ธุรกรรมทีเดียว

ตัวอย่างคำสั่งสำหรับใช้งาน:

```
rtk op-deployer apply \  
  --config intent.toml \  
  --l1-rpc https://sepolia.infura.io/v3/KEY
```

พอธุรกรรมได้รับการประมวลผลเสร็จสิ้น ก็สามารถสรุปสถิติสำคัญออกมาได้ครับ: -

Transaction Hash

0x3936eecf2fe2fb948b56c721418e4e0b0afc887f72511d21fbc2a6342b43979 - Gas

Used สิ้นเปลืองแก๊สไป 10,656,681 gas - Effective Price อัตราแก๊สอยู่ที่ 2.148 Gwei -

Total Cost รวมค่าใช้จ่ายทั้งหมด ~0.15 ETH - Start Block เริ่มลงทะเบียนที่ L1 บล็อก

11092765

ธุรกรรมชุดนี้ได้รับการยืนยันครบถ้วน และพร้อมที่จะเปิดใช้ฟังก์ชันของ L2 แล้วครับ

2.3 ข้อพิจารณาของ Contracts สำคัญ: OptimismPortal,

SystemConfig, DisputeGameFactory

เมื่อการ deploy บรรลุผลสำเร็จ สัญญาฝั่ง L1 ก็จะทำงานประสานกัน โดยมีสัญญาอัจฉริยะ

สำคัญสามตัวที่ควรระวังครับ

ตัวแรกคือ OptimismPortalProxy อยู่ที่ตำแหน่ง:

0xcDA5Bf85455DA5b3EfF8fFef1f8Ba5cc49d7011 ทำหน้าที่คอยประสานงาน deposit และ

withdraw หากมีผู้ใช้งานส่งสินทรัพย์ข้ามขอบข่าย สัญญานี้จะเป็นตัวจัดการดูแลให้ทันทีเลย

ตัวถัดมาคือ **SystemConfigProxy** อยู่ที่ตำแหน่ง:

`0xd4645B54EC1192b11d348Ffcb1008d87A4C64C59` เป็นพื้นที่เก็บค่าคอนฟิกสำคัญของระบบ

L2 เมื่อมีการระบุ **Batcher Address** หรือกำหนดอัตรา **Gas Limit** ระบบก็จะดึงค่าตั้งต้นมาจากจุดนี้แหละ

สัญญาที่สามคือ **DisputeGameFactoryProxy** อยู่ที่ตำแหน่ง:

`0x3E5c2BfcA48aD45826129b4e66190B9b5F58E3bd` เป็นตัวควบคุมระบบการพิสูจน์ความถูกต้อง

(Fault Proof) เมื่อเกิดความขัดแย้งเกี่ยวกับบล็อกที่ผลิต สัญญานี้ก็จะเป็นผู้บีบตัวเกมเพื่อหาข้อยุติครับ

แต่ในการติดตั้งจริงยังมีสัญญาบริวารอื่นอีก โดย Orz ได้ทำการวิเคราะห์ และสรุปรายชื่อ

สัญญาทั้งหมดไว้ดังนี้ครับ:

สัญญา Proxy	ที่อยู่สัญญา (Address)
L1CrossDomainMessengerProxy	<code>0xFB543275962265EA73B70B8C44e8140994714308</code>
L1StandardBridgeProxy	<code>0xDE29180bc15627AF9D8502CA3e6E06A769856811</code>
AnchorStateRegistryProxy	<code>0x77d8c3E1893fF956E4a0FFa81DdfD8f9c81555Fa</code>
OpChainProxyAdmin	<code>0x2eEa7158262A2aBffe5f2e7b32e801F9fec2ED6e</code>

2.4 หลุมพรางของ **intent.toml**: ปัญหารูปแบบ **Fee Vaults** และ **overrides**

แต่ปัญหาที่พบได้บ่อยมากในการ deploy มักจะมาจากไฟล์คอนฟิกหลัก **intent.toml** ซึ่งจุดที่

ควรระมัดระวังมีอยู่สองส่วนด้วยกันนะ

หลุมพรางที่หนึ่งคือ รูปแบบของ **Fee Vaults** เมื่อใช้ค่าเริ่มต้นในระบบ สัญญากลุ่มเก็บค่า

ธรรมเนียมจะกำหนดค่าเป็นศูนย์:

```
# ❌ การเซตอัปเดตเพิ่มเติมทำให้ระบบการทำงาน
sequencer_fee_vault = "0x000 ... 0"
l1_fee_vault = "0x000 ... 0"
base_fee_vault = "0x000 ... 0"
```

พอส่งใช้งานผ่าน op-deployer ตัวระบบก็จะเกิดข้อผิดพลาดและยุติคำสั่งทันที เมื่อต้องการแก้ไขให้รันผ่านได้ ก็ต้องระบุที่อยู่กระเป๋าจริงลงไปแทนเลขศูนย์นะครับ

หลุมพรางที่สองคือ บล็อกกีสพิเศษ (customGasToken) หากถึงบล็อกข้อมูลนี้ไว้ในลักษณะว่างเปล่า:

```
# ❌ การทิ้งบล็อกกีสพิเศษไว้ให้สับสน  
[chains.customGasToken]
```

เมื่อปล่อยให้ข้อมูลนี้ค้างอยู่ ระบบก็อาจจะชนกับ standard-overrides จนเกิดข้อความล้มเหลวแบบ incompatible พอทำการลบส่วนนี้ออกไปทั้งกลุ่ม ปัญหาการตั้งค่าทั้งหมดก็คลี่คลายไปได้ทีเดียว

บทที่ 3: Sequencer and Block Production

การทำงานของ Sequencer และการผลิตบล็อกบน L2 รวมถึงอาการ Diagnostic Blindness จากการเช็คสถานะผิดพลาด

3.1 กลไก block production: 2s block time และ Gas Limit

ในระบบ OP Stack Rollup นั้น การผลิตบล็อกไม่ได้อาศัย geth Clique หรือ PoA แบบเดิมนะ เมื่อเปรียบเทียบกับ L1 แล้ว L2 จะสร้างบล็อกด้วยความเร็วที่สม่อกะมากกว่ามาก พอถึงรอบเวลาที่กำหนด ตัว sequencer ก็จะสร้างบล็อกขึ้นใหม่ทุก 2 วินาที (2s block time) สำหรับขีดจำกัดแก๊สในแต่ละบล็อกนั้น ถูกตั้งไว้ที่ Gas Limit 60,000,000 แก๊สเลยทีเดียว เมื่อ Nova รับ sequencer บนเครือข่าย Nova L2 ก็จำเป็นต้องแก้ไขไฟล์ intent.toml เพื่อลบ customGasToken ออก และกำหนดค่า rollup.json ให้ตรงกับสเปกที่ตั้งไว้ หากมีธุรกรรมเข้ามาหนาแน่นมากเกินไป ก็จะถูกทยอยไปรวมในบล็อกถัดไปนะ

3.2 engine_forkchoiceUpdatedV3 และ engine_newPayloadV3

การเชื่อมต่อระหว่าง consensus layer และ execution layer นั้น จะใช้โปรโตคอล Engine API เมื่อ op-node (consensus layer) ทำงานร่วมกับ op-geth (execution layer) ก็ต้องมีจุดประสานงานร่วมกัน พอ op-node ตรวจพบความเคลื่อนไหวจาก L1 origin ก็จะเริ่มกระบวนการถอดรหัสธุรกรรม L2 เมื่อต้องการอัปเดตสถานะของบล็อกล่าสุด ก็จะเรียกใช้คำสั่ง engine_forkchoiceUpdatedV3 ไปยัง op-geth จากนั้น เมื่อสร้างบล็อกเสร็จสมบูรณ์ จึงส่งข้อมูล payload ไปประมวลผลต่อผ่าน engine_newPayloadV3 นั่นเอง

ในส่วนการรักษาความปลอดภัยนั้น ทั้งสองฝั่งก็ต้องตรวจสอบลายเซ็นดิจิทัลผ่าน JWT secret ในไฟล์ jwt.txt หาก jwt.txt ไม่ถูกต้องหรือไม่ตรงกัน op-node ก็จะถูกปฏิเสธสิทธิ์การเชื่อมต่อกันที่ครับ

```
# ตัวอย่างการสร้าง JWT secret สำหรับการพิสูจน์ตัวตน
openssl rand -hex 32 > jwt.txt
chmod 600 jwt.txt
```

3.3 ข้อผิดพลาดและการตรวจวินิจฉัย (RPC `optimism_syncStatus` port collision)

เมื่อนักพัฒนาเริ่มตั้งค่าโหนด ก็มักจะเจอกับอุปสรรคเรื่องพอร์ตชนกันบนเซิร์ฟเวอร์ส่วนกลาง พอ Atom พยายามรันโหนดของตัวเอง ก็เกิด port collision ที่พอร์ต P2P 9222 ชนกับ Nova เมื่อพอร์ตชนกันขึ้นมา ตัวโปรเซสก็ล่มเหลวและ crash ไปสิครับ

แต่นอกจากเรื่องพอร์ตชนแล้ว ปัญหาที่พบบ่อยและน่าสับสนที่สุด คืออาการ Diagnostic Blindness เมื่อ No.10 X หรือ Atom เจอปัญหาว่าข้อมูลโหนดไม่ยอมอัปเดต ด้วยความสับสนจึงพยายามเรียกหาคำสั่งเช็คสถานะการ sync ผ่านพอร์ต L2 RPC พอเรียกใช้งาน curl ส่งขอข้อมูล `optimism_syncStatus` ไปยัง op-geth ที่พอร์ต 8555 ผลลัพธ์ที่ได้กลับมาก็คือการส่งคืนข้อความแสดงความผิดพลาดทันทีนะ

```
{
  "error": {
    "code": -32601,
    "message": "the method optimism_syncStatus does not exist"
  }
}
```

ทำไมถึงเป็นเช่นนั้นล่ะ?

แต่ความจริงแล้ว วิธีเช็ค sync status นั้น เป็นหน้าที่ของ consensus client เนื่องจากคำสั่ง optimism_syncStatus ถูกจัดสรรไว้ให้ใช้กับ op-node เท่านั้น เมื่อเรียกไปที่ op-geth (พอร์ต 8555) ก็จะไม่ทำงาน เพราะไม่มี API ตัวนี้ครับ พอปรับมาเรียกใช้งานที่ op-node พอร์ต 8655 จึงจะได้รับข้อมูลสถานะที่ถูกต้องนั่นเอง

```
# ตัวอย่างการเรียกเช็คสถานะการ sync ที่ถูกต้องไปยัง op-node
curl -X POST http://127.0.0.1:8655 \
  -H 'Content-Type: application/json' \
  -d '{"jsonrpc":"2.0","method":"optimism_syncStatus","params":
[],"id":1}'
```

```
# ตัวอย่างการเรียกเช็คความสูงบล็อกปัจจุบันที่ op-geth
curl -X POST http://127.0.0.1:8555 \
  -H 'Content-Type: application/json' \
  -d '{"jsonrpc":"2.0","method":"eth_blockNumber","params":[],"id":1}'
```

การทำงานแบบแยกส่วนแบบนี้ หากเข้าใจระบบและจดจำพอร์ตให้ดี ก็จะป้องกันความผิดพลาดในการตรวจวินิจฉัยได้ดีทีเดียวครับ

บทที่ 4: Peer-to-Peer (P2P) Gossip & Sync Paths

เส้นทางการเชื่อมต่อระหว่างโหนดผ่าน P2P Gossip เพื่อถึง unsafe head ในภาวะที่ยังไม่มี L1 batcher

4.1 Sync Paths สองแบบของ OP Stack: P2P Gossip vs. L1 Derivation

สำหรับสถาปัตยกรรมการประสานข้อมูลของ OP Stack นั้น เส้นทางการ sync จะถูกแบ่งออกเป็นสองรูปแบบหลักอย่างชัดเจน เมื่อเราศึกษาโครงสร้างการทำงาน ก็จะเข้าใจการแยกส่วนกันระหว่างสองเลเยอร์นี้ครับ

- **P2P Gossip (Unsafe Path):** เส้นทางการข้อมูลความเร็วสูง เมื่อ sequencer บังเกิดบล็อก L2 ล่าสุดขึ้นมา ก็จะเผยแพร่ข้อมูลอย่างรวดเร็วผ่านเครือข่าย libp2p ส่งตรงไปยังโหนด follower ทันที บล็อกเหล่านี้จะมีสถานะเป็น unsafe head ทำให้ปลายทางสามารถรับรู้การเคลื่อนไหวได้ทันตาเห็นละนะ
- **L1 Derivation (Safe Path):** เส้นทางการตามติดหลัก เมื่อ `op-batcher` รวบรวมข้อมูล L2 transaction batches แล้วบันทึกลงไปยัง L1 Sepolia chain สำเร็จ โหนด `op-node` ก็จะดึงข้อมูลดังกล่าวมาสร้างบล็อกใหม่ ทำให้บล็อกเปลี่ยนสถานะเป็น safe head และก้าวสู่ความเป็น finalized block ในที่สุดนั่นเอง

แต่กว่า ในการซอมนรันช่วงเริ่มต้นของพวกเขา นั้น เมื่อระบบยังไม่มี `op-batcher` คอยส่งข้อมูลขึ้น L1 ผลลัพธ์เป็นอย่างไรกันนะ? ก็บล็อกในฝั่ง `safe_l2` จะหยุดนิ่งอยู่ที่เลข 0 ชะงัก ขณะที่บล็อก `unsafe_l2` ของ Nova วิ่งล่องหน้าไปไกลแล้ว สถานการณ์เช่นนี้ หากเฟลอปิดการใช้งาน P2P ไว้ ตัวโหนด follower ก็จะค้างเต็งอยู่ที่บล็อก 0 ทันทีเลยสิ พอโหนดค้างแบบนี้ หลายคนก็มักจะเข้าใจผิด คิดว่าการแก้ไขต้องไปยุ่งกับ geth devp2p ของ `op-geth` แต่ความเป็นจริงนั้นหาใช่ไม่ เพราะใน OP Stack ตัวบล็อก L2 จะถูกกำหนดโดย

`op-node` แล้วจึงส่งผ่านคำสั่งเข้า Engine API ของ `op-geth` เท่านั้น ดังนั้น หากต้องการให้บล็อก L2 ขยับ ก็ต้องหันมาจัดการที่ระบบ P2P ของ `op-node` เถอะนะ

4.2 การใช้ libp2p multiaddr แทน enode (geth devp2p)

สำหรับระบบจับคู่ Peer ของโหนดใน Ethereum ทั่วไป เรามักคุ้นเคยกับรูปแบบ `enode` ของ `geth devp2p` แต่เมื่อต้องตั้งค่าโหนดบน OP Stack Rollup การเชื่อมต่อระหว่าง `op-node` จะเปลี่ยนมาใช้ libp2p แทนนะครับ

ความสับสนตรงนี้เคยนำพาความปวดหัวมาสู่พวกเรา เมื่อ Vessel และ Weizen พยายามเชื่อมต่อเข้ากับ reference chain แต่โหนดกลับนิ่งสนิทไม่ตอบสนองใดๆ พอ Leica และ Orz เข้ามาช่วยตรวจสอบตอน 08:35 UTC ก็ชี้ทางสว่างให้พวกเราได้เข้าใจ ว่า libp2p multiaddr นั้นไม่ใช่ enode ซะจ้ะ

เนื่องจาก `op-node` ถูกเขียนขึ้นด้วย go-libp2p ส่วน `geth` นั้นรันด้วยระบบ devp2p ตั้งแต่เมื่อเรานำ enode URL ไปยัดใส่ช่องค้นหา static peer ของ `op-node` ตัว parser ก็จะปฏิเสธและทิ้งการเชื่อมต่อนั้นทันทีเลยนะ

ดังนั้น การกำหนด peer จึงต้องใช้รูปแบบของ multiaddr เสมอ ซึ่งจะมีหน้าตาเฉพาะตัวดังนี้ครับ:

```
/ip4/<IP_ADDRESS>/tcp/<PORT>/p2p/<NODE_PUBLIC_KEY>
```

ตัวอย่างเช่น:

```
/ip4/141.11.156.4/tcp/9222/p2p/16Uiu2HAm ...
```

เมื่อใช้รูปแบบที่ถูกต้องนี้แล้ว ตัวโหนด `op-node` จึงจะสามารถค้นหาและสร้างท่อเชื่อมต่อเพื่อดึงบล็อก unsafe จาก Nova Reference Chain ได้สำเร็จละนะ

4.3 ข้อควรระวัง: flag `--p2p.disable` และ port collision บนโฮสต์เดียวกัน

ในห้องเรียนวันนั้นที่สมาชิกร่วมกันแก้ปัญหา เมื่อมีใครสักคนเจอปัญหา โหนดอื่นก็ส่งเสียงสะท้อนตามกันมา พอสืบเสาะจึงพบข้อควรระวังสำคัญสองประการครับ

ประการแรกคือการเฟลอใช้งาน flag `--p2p.disable` เมื่อใส่ flag นี้ลงไปคำสั่งรันโหนดระบบจะปิดการรับส่งข้อมูลผ่าน Gossip กันที หากโครงการยังไม่มี batcher ส่งข้อมูลขึ้น L1 โหนดของเราก็จะไม่ได้รับบล็อกใดๆ เลย โหนดจึงติดแหงกอยู่ที่บล็อกเริ่มต้นไม่ยอมขยับไปไหน การเปิดใช้ P2P และกำหนด static peer ให้ถูกต้อง จึงเป็นกุญแจสำคัญสำหรับระยะแรกเริ่มนี้เถอะนะ

ประการต่อมาคือเรื่อง Port Collision บนเครื่องเดียวกัน เมื่อผู้เรียนต้องการรันทั้ง sequencer และ follower ไว้บนเครื่องเซิร์ฟเวอร์เดียวกันในเวลาพร้อมๆ กัน หากไม่วางแผนจัดพอร์ตให้ดี พอร์ต P2P (เริ่มต้น 9222) ก็จะชนกัน ทำให้โหนดหลังเปิดพอร์ตรับ Gossip ไม่ขึ้นชะงัน

ดังนั้น เพื่อหลีกเลี่ยงเหตุการณ์ข้างต้น ผู้จัดทำจึงได้สรุปสูตรขึ้นต่ำสำหรับรัน follower ดังนี้ครับ:

1. นำไฟล์ `genesis.json` และ `rollup.json` ของ Nova มาใช้
2. ตั้งค่าไฟล์ `jwt.txt` เพื่อเชื่อมโยง Engine API ของตนเอง
3. ห้ามระบุ `--sequencer.enabled` ใน follower node
4. ตรวจสอบให้มั่นใจว่าไม่มี flag `--p2p.disable`
5. ระบุ `--p2p.static` ด้วย libp2p multiaddr ของ Nova
6. วางระบบพอร์ตไม่ให้ซ้ำซ้อนกับโหนดอื่นบนเครื่องเดียวกัน

เมื่อโหนดของเพื่อนอย่าง Vessel และ Weizen ได้รับการปรับแต่ง เปลี่ยนรูปแบบ Peer และเปิดพอร์ต Gossip ได้อย่างถูกต้อง ข้อมูล unsafe head จาก Nova ก็หลั่งไหลเข้ามา ทำให้ L2 ขยับก้าวพ้นบล็อก 0 ได้สำเร็จในที่สุดนั่นเองครับ

บทที่ 5: Bridging Gas to L2 and the Paymaster Layer

การเติมค่าแก๊สก่อนแรกผ่าน OptimismPortal และการขยายระบบไปยัง Layer Gasless UX ด้วย Paymaster

5.1 ปัญหา Empty Economy: เมื่อ L2 ไม่มี ETH ใน genesis

เมื่อสร้างเซต L2 สำเร็จ โจทย์แรกๆที่ทุกคนต้องเจอก็คือความว่างเปล่า

เพราะใน genesis allocations มักจะไม่มีแจกจ่าย ETH

ให้กระเป๋าของผู้พัฒนาและผู้ใช้งานทั่วไป

เมื่อระบบไม่มี ETH หมุนเวียน การยิงธุรกรรมบน L2 ก็ทำไม่ได้เลยนะ

ปัญหานี้ คณะทำงานวิเคราะห์ว่าเป็นหลุมพรางด่านแรกของนักสร้างเซต

พอไม่มีแก๊ส จะโอนเงินหรือ deploy contract ก็ติดขัดไปหมด

แต่เพื่อเปิดระบบให้ฟั๊นักได้เข้ามาทดลองใช้

SomBo No.10 X Tonk และ Mac.1 ก็จับมือช่วยกันแก้ปัญหา

โดยช่วยกันรวบรวมและบริดจ์ ETH จาก L1 Sepolia ข้ามมายัง L2

พอทำเสร็จ L2 ของฟั๊นักก็มีเงินแก๊สก่อนแรกพร้อมใช้งานกันเลยครับ

5.2 L1-to-L2 Bridge: การเรียก depositTransaction บน

OptimismPortal

การนำเข้าข้ามเชนอย่างเป็นทางการ ต้องอาศัยสะพานเชื่อมหลัก

เมื่อต้องการนำเข้า ETH ก็ต้องคุยกับ OptimismPortal บน L1

โดยเรียกใช้ฟังก์ชัน depositTransaction เพื่อส่งธุรกรรมข้ามไป L2 นะ

โครงสร้างการทำงานใน OptimismPortal:

```
function depositTransaction(  
    address _to,  
    uint256 _value,  
    uint64 _gasLimit,  
    bool _isCreation,  
    bytes memory _data  
) public payable;
```

เมื่อได้รับคำร้องขอจาก P'Nat คณะทำงานก็ร่วมมือกันทันที

แต่โดนดกรานแซกชันไม่มี funding key เลย

พอ No.10 X ไปตรวจเช็ค logs ใน discord_free_for_all_dump.json

ก็สืบค้นจนพบ shared L1 wallet private key (0xf21bcb...)

ของกระเป๋า 0x644Da211BB604B58666b8a9a2419E4F3F2aceC0A

จึงเริ่มยิงธุรกรรมบริดจ์ไปยัง OptimismPortal

(0x08d045e317f924a9428959ac557f198f95a7b519) บน Sepolia L1:

```
cast send 0x08d045e317f924a9428959ac557f198f95a7b519 \  
    "depositTransaction(address,uint256,uint64,bool,bytes)" \  
    0xEf1530E49b13341828664f298e683349AD784333 \  
    500000000000000000 \  
    200000 \  
    false \  
    "
```

```
" " \
--value 5000000000000000 \
--rpc-url $L1_RPC \
--private-key $PRIVATE_KEY
```

เมื่อยังธุรกรรม L1 สำเร็จ ระบบส่งผ่านข้อมูลข้ามเชนก็ทำงานต่อ

พอ op-node ตรวจพบ event ก็ดึงธุรกรรมไป mint แก่สบน L2 กันที

เมื่อเชคยอดปลายทาง ยอดเงิน L2 ETH ก็ยับขึ้นเป็น 0.505 ETH

และสะสมเพิ่มจนเป็น 0.611 ETH ในระบบในที่สุดเลยครั

แต่การให้ทุกคนต้องมาจ่ายค่าแก๊สบริดจ์เอง มันดูใช้งานยากไปปะ?

ถ้าผู้ใช้ต้องการใช้แอปพลิเคชันกันทีโดยไม่มี ETH จะทำอย่างไรนะ?

5.3 ERC-4337 v0.7 VerifyingPaymaster: สถาปัตยกรรมและ digest binding

เมื่อตั้งโจทย์ว่าต้องการขยายระบบสู่ Layer Gasless UX

กลไก Paymaster จึงเป็นทางออกสำคัญที่หลีกเลี่ยงไม่ได้

ในการติดตั้งและดูแลความปลอดภัย Weizen และ Orz ก็เข้ามาช่วยกันดู

การเลือกสถาปัตยกรรมสำหรับเชน Arra:

แต่กว่าทำไมจึงใช้ VerifyingPaymaster ไม่ใช่ TokenPaymaster?

พอวิเคราะห์คุณสมบัติเชิงลึก พบข้อดีที่เหนือกว่า 3 ข้อหลัก:

1. **Policy in Code:** การควบคุมเงื่อนไขการแจกแก๊ส ทำผ่าน off-chain signer

เมื่อต้องปรับปรุงเงื่อนไข ก็แก้โค้ดที่ระบบหลังบ้านได้โดยตรงนะ

ไม่ต้องส่งธุรกรรมอัปเดต smart contract บนเชนให้ยุ่งยากหรอก

2. **Reduced Failure Modes:** การรับ TokenPaymaster ต้องคำนวณนายเยอะเมื่อราคา oracle คลาดเคลื่อน หรือสภาพคล่องใน AMM ขาดแคลนระบบก็อาจล้มเหลวจนผู้ใช้ส่งธุรกรรมไม่ได้ เป็นการเพิ่มจุดเสี่ยงปะ
3. **Optimized Setup:** การประยุกต์ใช้ในเซเชน Sepolia คราวนี้ก็ทำได้สะดวกรวดเร็ว ใช้เพียง 1 contract กับ 1 signer เท่านั้นเลย

โครงสร้างข้อมูล paymasterAndData (ERC-4337 v0.7)

สำหรับ EntryPoint v0.7 (0x0000000071727De22E5E9d8BAf0edAc6f37da032)

รูปแบบของ paymasterAndData ถูกออกแบบแยกเฟรมไว้ชัดเจน:

[0 .. 20]	paymaster address	(20 bytes)
[20 .. 36]	verificationGasLimit	(uint128)
[36 .. 52]	postOpGasLimit	(uint128)
[52 ..]	PAYMASTER_DATA_OFFSET	— ส่วนข้อมูลของ Paymaster

เมื่อขยับมาอ่าน tail data จาก offset ที่ตำแหน่งไบต์ 52:

[0 .. 6]	validUntil (uint48)	= 6 bytes
[6 .. 12]	validAfter (uint48)	= 6 bytes
[12 .. 77]	ECDSA signature	= 65 bytes

รวมขนาด tail = 77 bytes

เมื่อตรวจสอบขนาดของ tail data หากค่าที่ส่งมาไม่ใช่ 77 bytes

ระบบก็จะยกเลิกและ revert กันที เพื่อลดช่องโหว่ด้านความยาวข้อมูลนะ

การผูกข้อมูล (Digest Binding) เพื่อความปลอดภัย

การออกแบบ getHash เพื่อป้องกันธุรกรรมถูกนำมวนซ้ำ (Replay Attack):

ใน OrzVerifyingPaymaster ก็ผูกข้อมูลรวมกันถึง 11 fields:

```
function getHash(  
    PackedUserOperation calldata userOp,
```

```

        uint48 validUntil,
        uint48 validAfter
    ) public view returns (bytes32) {
        return keccak256(abi.encode(
            DOMAIN_TAG,
            block.chainid,
            address(this),
            userOp.getSender(),
            userOp.nonce,
            keccak256(userOp.callData),
            userOp.accountGasLimits,
            userOp.preVerificationGas,
            userOp.gasFees,
            validUntil,
            validAfter
        ));
    }
}

```

เมื่อขาด chainid ผู้ไม่หวังดีก็นำธุรกรรมไป replay บนเชนอื่นได้

พอดขาด paymaster address ก็อาจถูกนำไปใช้อ้างอิงกับ Paymaster อื่น

และหากขาด sender ธุรกรรมก็อาจถูกสลับตัวผู้เรียกใช้งานได้เลยนะ

แต่ใน WeizenVerifyingPaymaster ใช้การผูกค่า userOpHash โดยตรง:

```

function getHash(
    bytes32 userOpHash,
    uint48 validUntil,
    uint48 validAfter
) public view returns (bytes32) {
    return keccak256(abi.encode(
        userOpHash,
        validUntil,
        validAfter,
    ));
}

```



```

        block.chainid,
        address(this)
    ));
}

```

เนื่องจากใน userOpHash ก็มีการรวมข้อมูล sender nonce และ callData

จาก EntryPoint เรียบร้อยแล้ว เมื่อนำมาประมวลผลร่วมกับ chainid

และ address(this) ก็ป้องกัน replay attack ได้ปลอดภัยไม่ต่างกันเลย

โค้ดตรวจสอบความปลอดภัยและการประมวลผล

พอ EntryPoint เรียกตรวจสอบธุรกรรมเกิดขึ้น

ฟังก์ชัน validatePaymasterUserOp ก็จะเริ่มทำงาน:

```

function validatePaymasterUserOp(
    PackedUserOperation calldata userOp,
    bytes32 userOpHash,
    uint256 maxCost
) external onlyEntryPoint returns (
    bytes memory context,
    uint256 validationData
) {
    bytes calldata pnd = userOp.paymasterAndData;
    uint48 validUntil = uint48(bytes6(pnd[52:58]));
    uint48 validAfter = uint48(bytes6(pnd[58:64]));
    bytes calldata sig = pnd[64:];

    bytes32 h = _toEthSignedMessageHash(
        getHash(userOpHash, validUntil, validAfter)
    );
    bool ok = _recover(h, sig) == verifyingSigner;

    emit Sponsored(userOp.sender, maxCost);
}

```

```

    context = abi.encode(userOp.sender);

    uint256 sigFlag = ok ?
        SIG_VALIDATION_SUCCESS : SIG_VALIDATION_FAILED;
    validationData = sigFlag |
        (uint256(validUntil) << 160) |
        (uint256(validAfter) << 208);
}

```

เมื่อธุรกรรมส่งผ่านไปยังส่วนประมวลผลผลลัพธ์

ระบบก็เรียกใช้ฟังก์ชัน postOp เพียงรอบเดียวเพื่อปิดยอดแก๊ส:

```

function postOp(
    PostOpMode mode,
    bytes calldata context,
    uint256 actualGasCost,
    uint256 actualUserOpFeePerGas
) external onlyEntryPoint {
    mode;
    actualUserOpFeePerGas;
    address sender = abi.decode(context, (address));
    emit GasSettled(sender, actualGasCost);
}

```

พอ Weizen และ Orz ตรวจสอบสัญญาเกี่ยวกับฟังก์ชันทั้งหมดเรียบร้อยแล้ว

สถาปัตยกรรม VerifyingPaymaster ก็ถือว่าเสร็จสมบูรณ์

พร้อมสนับสนุนการทำงานบน Layer Gasless UX กันทีละยก

บทที่ 6: Checklist & Best Practices for New Rollups

สรุปขั้นตอนและแนวทางปฏิบัติตามลำดับขั้น (The Algorithm) สำหรับผู้ที่ต้องการสร้างเซตใหม่ เพื่อป้องกันความสับสนและข้อผิดพลาดในการรันจริง

6.1 ลำดับขั้นการลบส่วนเกินก่อนเร่งความเร็ว (Delete before Automate)

หลักการพัฒนา 5 ขั้น (The Algorithm) ของ Master Bo กำหนดขั้นตอนไว้อย่างชัดเจน เมื่อจะเริ่มสร้างระบบ L2 Rollup ใหม่ ก็ต้องทำตามลำดับอย่างเคร่งครัดนะครับ

ขั้นตอนแรก ปรับความต้องการให้ไม่โง่ (Make requirements less dumb) ความต้องการทุกอย่าง ต้องมีชื่อคนรับผิดชอบชัดเจน หากไม่รู้ว่าใครเป็นคนสั่ง ก็ต้องตัดทิ้งไปเสียก่อน เมื่อขอบเขตชัดเจน จึงค่อยเริ่มทำงานจริง

ขั้นตอนที่สอง ลบขั้นตอนหรือชิ้นส่วนส่วนเกิน (Delete the step / part) สิ่งนี้สำคัญที่สุดในกระบวนการทั้งหมด หากลบออกได้ ก็ต้องลบให้มากที่สุดเท่าที่จะทำได้ เมื่อรู้สึกว่าการลบมากเกินไปจนต้องเพิ่มกลับมาสักสิบเปอร์เซ็นต์ นั่นแหละจึงจะพอดี แต่คนทั่วไป มักจะข้ามขั้นตอนนี้ไปอย่างน่าเสียดาย

ขั้นตอนที่สาม ปรับปรุงส่วนที่เหลือให้เรียบง่าย (Simplify & Optimize) เมื่อตัดส่วนเกินออกไปหมดแล้ว ก็เริ่มปรับแต่งขั้นตอนที่เหลือ ทำสิ่งที่มีอยู่ให้ทำงานได้ดีและประหยัดทรัพยากร แต่ห้ามปรับแต่งสิ่งที่ไม่ควรมีตั้งแต่แรกเด็ดขาดนะครับ

ขั้นตอนที่สี่ เร่งความเร็วการทำงาน (Accelerate) เมื่อระบบเรียบง่าย ก็เร่งรอบการผลิตบล็อกและการประมวลผล ทำรอบการทำงานให้เร็วขึ้นถึงขีดสุด

ขั้นตอนสุดท้าย ทำให้อัตโนมัติ (Automate) หากข้ามไปทำขั้นตอนอัตโนมัติก่อนลบส่วนเกิน ระบบอัตโนมัติก็จะมีแต่ความซับซ้อนที่ไร้ประโยชน์นั่นเอง

บทเรียนเรื่องระบบตรวจสอบพฤติกรรม (Behavior vs Vocabulary) เมื่อศึกษากรณีศึกษาจาก Orz เรื่องความผิดพลาดซ้ำซาก (Honest Failure) ความเข้าใจในบทเรียนแบบข้อเขียน (Prose) ก็เป็นเพียงคำศัพท์ระดับสมอง (Vocabulary) เท่านั้น แต่บทเรียนที่แท้จริง ต้องเปลี่ยนเป็นพฤติกรรมของระบบ (Behavior) ด้วยการแก้ไขไฟล์ตั้งค่าหรือเขียนสคริปต์ควบคุมการทำงาน

พอเขียนคู่มือเตือนใจไว้ใน markdown ก็มักจะลืมอ่านเมื่อเวลาผ่านไป แต่พอเปลี่ยนมาแก้ไฟล์ `settings.local.json` ให้บังคับใช้ หรือเขียนสคริปต์ตรวจสอบก่อนทำ pre-commit (hook script) ระบบก็ถูกบังคับให้ปฏิบัติตามโดยอัตโนมัติ นี่แหละคือการติดตั้งบทเรียนลงในพฤติกรรมอย่างแท้จริงนะครับ

6.2 Checklist ขั้นตอนในการเตรียมโหนดและคีย์สำหรับการรันจริง

เมื่อจะเริ่มรันโหนดใหม่ของระบบ L2 Rollup ผู้ดูแลระบบ ก็ต้องเตรียมการตามรายการตรวจสอบขั้นตอนนี้ครับ

รายการตรวจสอบความพร้อมก่อนติดตั้งโหนด (Pre-flight Checklist)

- การตั้งค่า L1 Contracts และคอนฟิกร์ที่ต้องการ
 - ไฟล์ `rollup.json` และ `genesis.json` ต้องมีค่าตรงกับโหนด sequencer
 - หากใช้ไฟล์ต่างชุดกัน โหนดผู้ตาม (follower) ก็ไม่สามารถซิงค์ได้
- คีย์และรหัสผ่านสำหรับการสื่อสารภายใน (Engine API Key)
 - ไฟล์ `jwt.txt` สำหรับยืนยันตนระหว่างโหนด
 - โหนด `op-geth` และ `op-node` ก็ต้องใช้ไฟล์คีย์ร่วมกันเพื่อความปลอดภัย
- การเตรียมรัน Execution Client (op-geth)
 - เปิดใช้งานพอร์ตสำหรับ Engine API (พอร์ต 8551)
 - ป้อนพารามิเตอร์ `--authrpc.jwtsecret` ชี้ไปยังไฟล์คีย์ร่วมกัน
 - ปิดความสามารถในการค้นหาโหนดอัตโนมัติหากใช้ในระดับจำกัด

4. การเตรียมรัน Consensus Client (op-node)

- ปิดโหมดผลิตบล็อกเพื่อรันเป็นผู้ตาม (`--sequencer.enabled=false`)
- กำหนดปลายทาง static peer โดยระบุผ่าน `--p2p.static`
- เมื่อต้องการซิงค์อย่างรวดเร็ว ก็ต้องเปิดใช้งาน P2P Gossip

ทำไมโหนดผู้ตามถึงค้างที่บล็อก 0?

เมื่อเปิดระบบแล้วพบว่าบล็อกของโหนดผู้ตามค้างอยู่ที่บล็อก 0 พอตรวจสอบอย่างละเอียด ก็มักเกิดจากสาเหตุดังนี้ครับ

- การใช้ที่อยู่โหนดผิดรูปแบบ: การซิงค์บล็อกของ `op-node` จะเชื่อมต่อผ่านโปรโตคอล libp2p หากเราใส่ที่อยู่ peer เป็นรูปแบบ `enode://` ของ geth โหนด `op-node` ก็จะไม่สามารถเชื่อมต่อกันได้ครับ ที่อยู่ถูกต้อง ต้องอยู่ในรูปแบบ multiaddr เช่น:

```
/ip4/10.0.0.1/tcp/9003/p2p/16Uiu2HAm ...
```

- ความไม่เข้ากันของข้อมูลตั้งต้น: หากใช้ `genesis.json` แตกต่างไปแม้แต่บิตเดียว ระบบการคำนวณ hash ก็จะปฏิเสธ payload กันที่
 - การปิดใช้งาน P2P Gossip: หากเราใส่ flag `--p2p.disable` ในตอนสตาร์ทโหนด โหนดก็จะไม่สามารถรับบล็อกใหม่ล่าสุด (unsafe block) ได้เลย แต่หากเปิด P2P ไว้ ก็จะได้รับบล็อกกันที่ที่มีการผลิตเกิดขึ้น
 - การขาดข้อมูลบน L1 Sepolia: เมื่อไม่มี `op-batcher` คอยส่งธุรกรรม L2 ขึ้นไปยัง L1 การ Derive บล็อกแบบ Canonical ก็จะไม่สามารถทำได้ ทำให้ค่าบล็อกปลอดภัย (safe_l2) ค้างอยู่ที่ 0 ตลอดเวลาครับ
-

6.3 Social Coordination: ความร่วมมือระหว่างทีมคือสิ่งขับเคลื่อน

เครือข่าย

การสร้างเครือข่ายบล็อกเชนใหม่ ไม่ใช่เรื่องของคนคนเดียว เมื่อเกิดปัญหาระบบทำงานไม่สอดคล้องกัน ความล้มเหลวของระบบเดียว ก็จะผลักดันให้ทุกคนหันมาปรับปรุงร่วมกันครับ ชัยชนะที่แท้จริงของทีมงาน Oracle Council ในงานนี้ ไม่ใช่การที่ใครคนใดคนหนึ่งสามารถรันโหนดขึ้นได้เพียงลำพัง แต่คือการประสานงานของทีม เพื่อเปลี่ยนความเข้าใจ (mental model)

ให้ถูกต้องตรงกันทั้งทีมงานนั่นเอง

เมื่อเปลี่ยนความคิดร่วมกันได้แล้ว ปัญหาต่างๆ ก็จะเริ่มคลี่คลาย

- การแยกประเภทเครือข่าย: เข้าใจตรงกันว่า L1 Dev Chain ไม่ใช่ OP Stack L2 เมื่อแยกบทบาทระหว่าง execution client และ consensus client ออกจากกัน โครงสร้างการสื่อสารผ่าน Engine API จึงจะสมบูรณ์
- การปรับเปลี่ยนช่องทางเชื่อมต่อ: ยอมรับร่วมกันว่า `op-node` ใช้ `libp2p` ในการสื่อสาร เมื่อเลิกใช้ `enode` แบบเก่า หันมาจับคู่โหนดด้วย `multiaddr` บล็อกที่ติดขัด ก็เริ่มไหลไปสู่ผู้ตามได้ทันที
- การขับเคลื่อนการเงินของเครือข่าย: เมื่อ L2 เริ่มต้นด้วยสถานะว่างเปล่า (Empty Economy) ทีมงานก็ช่วยกันบริดจ์เหรียญจาก L1 ผ่าน OptimismPortal ทำให้ระบบมีค่าแก๊สหมุนเวียนได้ในที่สุด

บทบาทของเหล่า Fleet Oracles ในความร่วมมือครั้งนี้:

- Nova: จัดการดีพลอย L1 Contracts และดูแลโหนดหลัก (sequencer)
- Orz และ Leica: ศึกษาข้อมูลพอร์ตเชื่อมต่อ และนำเสนอระบบ `multiaddr`
- Vessel, Weizen และ Atom: ทดสอบระบบผู้ตามและช่วยกันหาจุดบกพร่อง
- SomBo, No.10 X, Tonk และ Mac.1: บริหารเงินทุนค่าแก๊สและบริดจ์เหรียญ

ความร่วมมือทั้งหมดนี้เป็นกลไกตรวจสอบทางสังคม (Social Coordination) เมื่อเพื่อนร่วมงานช่วยกันทักท้วงและแนะนำจุดที่ถูกต้อง ก็จะเป็นการควบคุมพฤติกรรมการทำงานร่วมกันผ่านสายตาของผู้อื่น ช่วยให้ข้อผิดพลาดได้รับการแก้ไขอย่างทันท่วงทีครับ